

Digital Twin of the Photolithography Step in Semiconductor Manufacturing

Technical Report

Prepared by: Connor Hatfield

Date: 5/1/26

Contents

Executive Summary	3
1. Introduction.....	4
2. System Architecture.....	4
2.1 Python Lot Generation Pipeline.....	5
2.2 AnyLogic Intake and Lot Creation	6
2.3 Process Flow Logic.....	7
2.4 Recipe-Specific Routing	7
2.5 Downtime Modeling	8
2.6 User Interface and Runtime Control	8
3. Validation.....	9
3.1 Data-Pipeline Validation.....	9
3.2 Logic Validation	9
3.3 Behavior Validation	9
3.4 Limits of Validation.....	10
4. Experiments and What-If Analysis	10
4.1 Baseline Scenario.....	10
4.2 Scenario 1: Increased Demand.....	10
4.3 Scenario 2: Recipe Mix Shift.....	11
4.4 Scenario 3: Increased Downtime Severity	11
4.5 Summary of Experimental Findings	11
5. Discussion	13
6. Conclusion and Future Work	13

Executive Summary

This project developed a digital twin of the photolithography step in semiconductor manufacturing using AnyLogic for discrete-event simulation and Python for external lot generation. The objective was to create a semi-realistic, interactive model that captures lot arrivals, queueing, recipe-specific routing, tool-level downtime, and user-adjustable operating conditions. Rather than hard-coding arrivals inside the simulation, the model receives lot data generated by an external Python script over a socket connection. This design better reflects how a production digital twin may ingest data from outside systems rather than relying solely on internal simulation sources.

The final model supports multiple photolithography tools, recipe-qualified routing, independent machine failures, and a user interface that allows runtime adjustment of selected input parameters. The model also tracks performance indicators such as queue length, work-in-process, throughput, and cycle time. Together, these capabilities make the twin suitable for evaluating what-if scenarios involving demand shifts, recipe mix changes, and equipment reliability changes.

The most important outcome of this project is not a claim of full industrial fidelity, but rather the successful creation of a functioning digital twin framework. The system demonstrates the end-to-end pipeline needed for a practical twin: data generation, data ingestion, simulation logic, runtime control, and scenario analysis. The model provides a solid foundation for future expansion into more realistic fab behavior, including richer photolithography process detail, maintenance schedules, additional constraints, and validation against real production data.

Photolithography Digital Twin

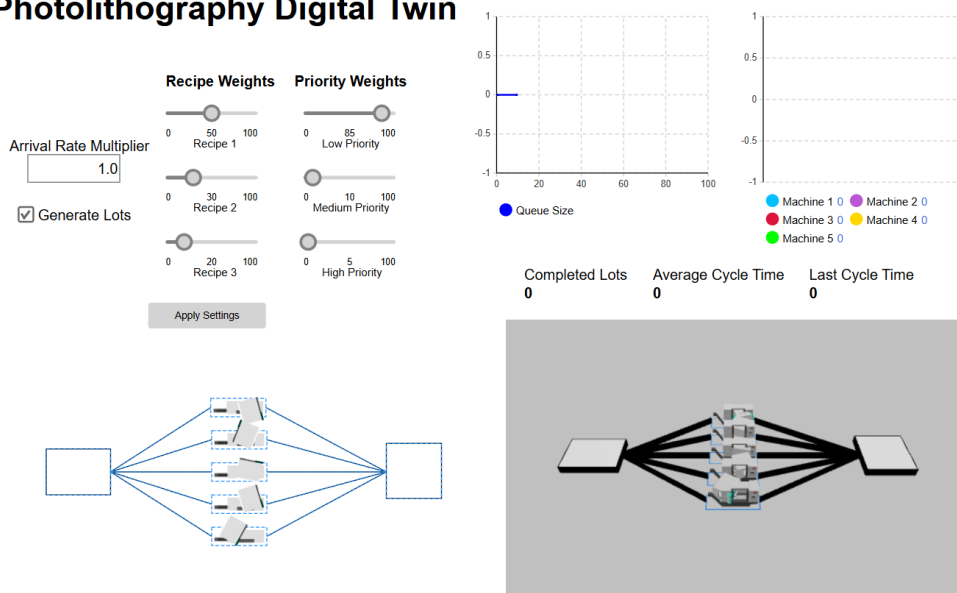


Figure 1: Landing page/Top-level UI

1. Introduction

Photolithography is one of the most critical and capacity-sensitive steps in semiconductor manufacturing. It often acts as a bottleneck because tools are expensive, recipes are specialized, and disruptions can propagate downstream throughout the fab. These characteristics make photolithography a strong candidate for digital twin development. A digital twin can support experimentation without interrupting production, help engineers understand the effect of failures and demand changes, and provide a platform for planning and optimization.

This project focused on building a semi-realistic digital twin of the photolithography step for a class setting. The model was intentionally scoped to balance realism and implementation time. It captures several core operational behaviors that matter in practice: dynamic lot arrivals rather than fixed deterministic arrivals, recipe-specific processing times, recipe-qualified routing to tools, queueing before tool assignment, independent tool downtime, and runtime user controls for scenario exploration.

The resulting system is not a full fab model and does not claim to reproduce all physical and chemical details of lithography. Instead, it represents the operational behavior of the step at the lot and tool level, which is appropriate for throughput, congestion, and reliability studies.

2. System Architecture

The digital twin consists of two main components: the Python lot generator and the AnyLogic simulation model. Python is responsible for creating lot records and sending them to AnyLogic over a TCP socket. AnyLogic acts as the simulation engine, receives incoming lot data, converts each record into a simulation entity, and routes that entity through the photolithography process.

System Architecture: Photolithography Digital Twin

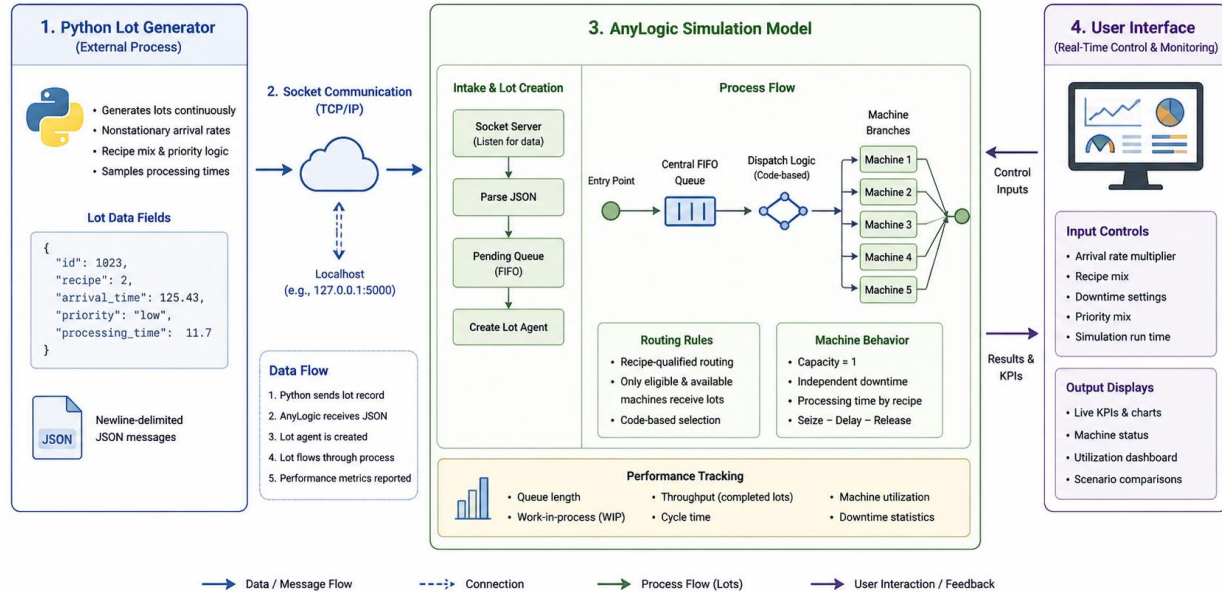


Figure 2: High-level system architecture diagram showing Python, socket communication, AnyLogic, and the user interface

This separation makes the architecture modular and closer to a real digital twin deployment, where data may come from external software rather than from internal simulation blocks alone.

2.1 Python Lot Generation Pipeline

The Python script generates lots continuously and sends them to AnyLogic as newline-delimited JSON messages over a local TCP socket. Each lot contains five fields: id, recipe, arrival_time, priority, and processing_time.

The arrival process is nonstationary. The script uses a repeating 60-minute schedule with arrival-rate levels of 4.0, 7.0, and 5.0 arrivals per minute over different parts of the cycle, then samples actual interarrival times from an exponential distribution based on the current rate. The default recipe mix is weighted as 35 percent recipe 1, 35 percent recipe 2, and 30 percent recipe 3. The default priority mix is 85 percent low, 10 percent medium, and 5 percent high. Processing times are sampled from recipe-specific ranges of 6 to 10, 9 to 14, and 12 to 18 time units for recipes 1, 2, and 3 respectively.

This design gives the twin several useful properties. First, it avoids the unrealistic assumption of constant arrivals. Second, it allows different operational stress to emerge depending on recipe composition. Third, it supports external control, since the generation logic can be modified without restructuring the AnyLogic model.

```
def generate_lot():
    global lot_id_counter

    config = get_runtime_config_snapshot()
    if not config["generatorEnabled"]:
        return None

    recipe = int(weighted_choice(config["recipeWeights"]))
    priority = weighted_choice(config["priorityWeights"])
    pt_low, pt_high = PROCESSING_TIME_BY_RECIPE[recipe]

    processing_time = random.uniform(pt_low, pt_high)
    if priority == "high":
        processing_time *= random.uniform(0.95, 1.05)

    lot_id_counter += 1

    return {
        "id": lot_id_counter,
        "recipe": recipe,
        "arrival_time": time.time() - start_time,
        "priority": priority,
        "processing_time": processing_time,
    }
```

Figure 3: Screenshot of the lot generator portion of the Python script

2.2 AnyLogic Intake and Lot Creation

On the AnyLogic side, a server socket listens for incoming data from Python. Incoming JSON messages are parsed into an intermediate data object, placed into a pending queue, and then converted into Lot agents by a polling event. The key simulation attributes stored in each lot include lot ID, recipe, arrival time, priority, and processing time.

This architecture proved more robust than trying to inject lots directly from the socket thread into the process model. The pending queue isolates asynchronous I/O from the simulation engine and creates a clean handoff between the external data stream and the AnyLogic event logic.

2.3 Process Flow Logic

Once created, each lot enters the main process flow. The final design uses an entry point for incoming lots, a central FIFO queue, code-based dispatch logic, one machine branch per tool, and a sink for completed lots.

An earlier attempt used a SelectOutput5 routing block, but this was replaced by custom dispatch logic because the block forced the final output to behave like a default fallback branch. That behavior conflicted with the requirement that the fifth machine accept only specific recipes. The final code-based dispatch design is better because it supports true central-queue control, recipe-qualified assignment, and easier extension for tool downtime.

Each machine has its own resource pool with capacity 1, so each tool behaves as an individual asset rather than as a shared abstract pool. That choice was important because the project eventually needed machine-specific downtime and machine-specific eligibility.

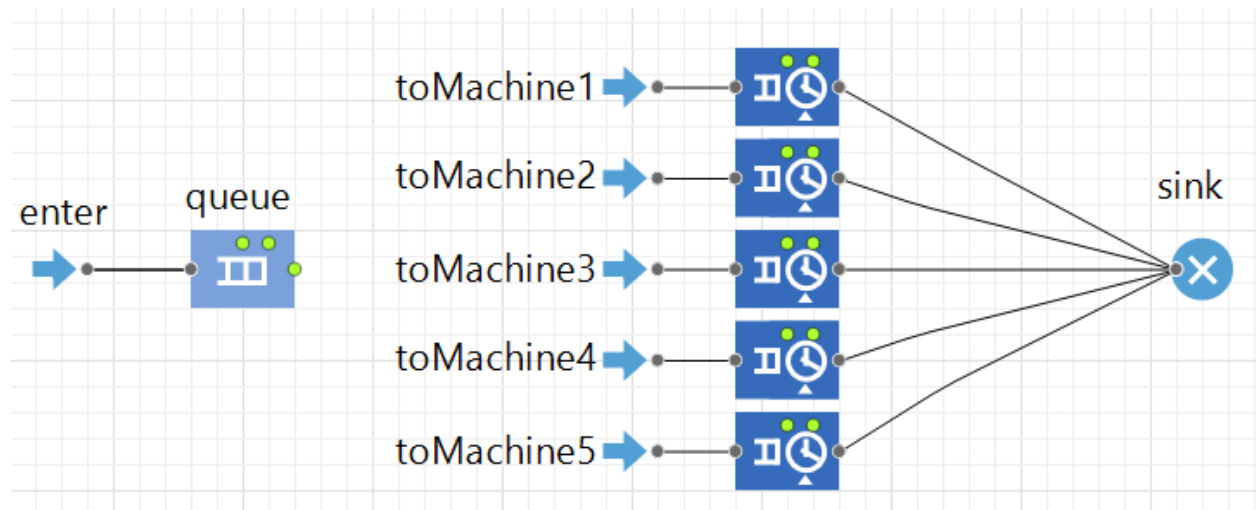


Figure 4: Process-flow showing entry, queue, machine branches, and sink

2.4 Recipe-Specific Routing

The model supports recipe-qualified routing. Lots are dispatched only to machines that are both eligible to run the recipe and currently available. This allows the twin to represent situations where not every tool can process every lot. It also creates more realistic congestion patterns, because demand can become concentrated on a subset of tools if the recipe mix shifts.

From a digital twin perspective, this is important because tool capability is one of the main sources of complexity in semiconductor manufacturing. A single step may not be limited by nominal tool count alone. It may be limited by the count of tools qualified for a specific recipe family.

2.5 Downtime Modeling

Independent downtime was implemented for each machine using AnyLogic downtime blocks configured as failure and repair tasks. The downtime logic is attached to the individual machine resource pools, not to the overall queue. This makes failures machine-specific, which is appropriate for a photolithography environment where individual tools may go down independently for repair or maintenance.

A key refinement in the dispatch logic was the addition of machine-up boolean flags. Without those flags, the simulation could still dispatch a lot toward a machine branch even while that machine was down, causing the lot to wait locally rather than at the central queue. The explicit availability flags made the dispatch logic downtime-aware and ensured that a machine was considered assignable only if it was both empty and operational.

2.6 User Interface and Runtime Control

The final model includes an AnyLogic user interface with controls that allow the user to change model inputs at runtime. These controls were intentionally placed in AnyLogic rather than in Python so the model feels like an interactive digital twin rather than a fixed back-end simulation.

User-adjustable controls include whether lot generation is enabled, an arrival multiplier, recipe mix weights, priority mix weights, and scenario settings related to machine behavior or demand. A second control path was added so that AnyLogic UI changes could be sent back to Python while still keeping Python responsible for generating lots. This preserved the basic architecture of the project while adding interactivity.

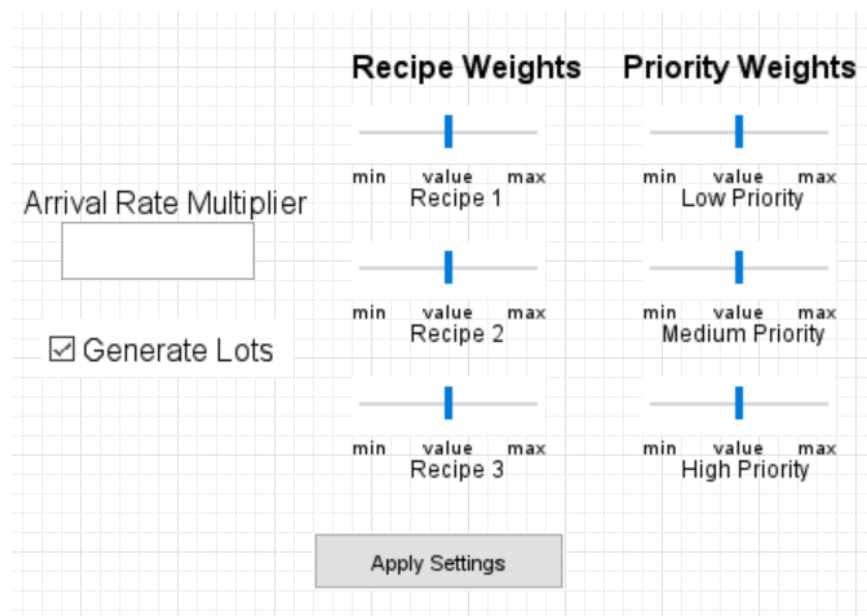


Figure 5: Screenshot of the UI controls panel in the AnyLogic editor. The view when running the simulation can be seen in Figure 1.

3. Validation

Because of the relatively small scale of the project, validation was performed primarily through structural and behavioral checks rather than formal industrial acceptance testing. The main question was not whether the model reproduces a particular real fab exactly, but whether it behaves consistently with the intended operational logic of a photolithography step.

Validation was conducted at three levels: data-pipeline validation, logic validation, and behavior validation.

3.1 Data-Pipeline Validation

The first requirement was to confirm that the Python-to-AnyLogic connection worked correctly. This was validated by confirming socket connection establishment, confirming that lots were generated and transmitted, confirming that each received message was parsed correctly, and confirming that lots were injected into the simulation with the correct attributes.

Console logs were used extensively during this stage. The logs showed that incoming lots were received, converted into simulation entities, placed into the queue, assigned to machines, and ultimately sent to the sink.

3.2 Logic Validation

The next step was to validate whether the simulation logic matched the intended behavior. This included checking that lots are not lost between intake and completion, lots enter the queue before processing, only eligible machines receive a given recipe, machines do not process more than one lot at a time, and downtime prevents new work from being assigned to unavailable tools.

Several issues discovered during development helped improve the model. For example, the original routing block design could not enforce independent conditions on all five outputs. Replacing it with code-based dispatch resolved that limitation. Another issue involved lots being sent toward machines that were down; adding machine-up state checks corrected that behavior.

These debugging steps improve confidence in the internal validity of the twin. The model now behaves consistently with the operational rules it was intended to represent.

3.3 Behavior Validation

Behavior validation focused on whether system-level outcomes made sense qualitatively. The model was expected to show queue growth under high demand or frequent downtime, lower congestion when demand is reduced, machine-specific loading differences under different recipe mixes, throughput reduction when tool availability is reduced, and longer cycle times when failures increase.

Observed simulation behavior was consistent with these expectations. Queueing increased under heavier load and tighter tool availability. Machines that were eligible for more recipes or more common recipes tended to experience higher usage. Downtime created visible reductions in effective capacity, particularly when it affected a tool that supported a constrained subset of recipes.

3.4 Limits of Validation

The main limitation is that validation was not performed against real manufacturing records such as dispatch logs, MES timestamps, tool state histories, or actual repair distributions. Therefore, the model should be treated as a structurally valid prototype twin rather than a production-certified representation of a real photolithography cell.

4. Experiments and What-If Analysis

This section presents the scenario structure that best fits the model. The metrics currently tracked in the simulation dashboard include queue length, work-in-process, completed lots, average cycle time, machine utilization or busy time, machine up or down status, and downtime counts and duration.

4.1 Baseline Scenario

The baseline scenario should use the default arrival schedule, recipe mix, and machine configuration. Under this setting, the twin establishes nominal queue, throughput, and cycle-time behavior.

Completed Lots	Average Cycle Time	Last Cycle Time	Average Queue Length
30	11.119	9.74	0.032
Most Utilized Machine	Bottleneck Evidence		
Machine 1	No strong bottleneck evidence under baseline conditions.		

Figure 6: Baseline results

4.2 Scenario 1: Increased Demand

In this scenario, the arrival multiplier is increased to 10x while the rest of the system remains unchanged. The purpose is to determine how sensitive the photolithography step is to increased lot release volume.

Completed Lots 66	Average Cycle Time 78.621	Last Cycle Time 158.99	Average Queue Length 67.823
Most Utilized Machine Machine 1	Bottleneck Evidence Sustained queueing with one highly utilized machine suggests a bottleneck.		

Figure 7: Increased lot load results

4.3 Scenario 2: Recipe Mix Shift

In this scenario, the recipe distribution is changed to emphasize one recipe family. The purpose is to evaluate how recipe-specific tool constraints affect system balance.

Completed Lots 53	Average Cycle Time 32.938	Last Cycle Time 51.062	Average Queue Length 5.023
Most Utilized Machine Machine 1	Bottleneck Evidence Sustained queueing with one highly utilized machine suggests a bottleneck.		

Figure 8: Focused recipe results

4.4 Scenario 3: Increased Downtime Severity

In this scenario, failure frequency or repair duration is increased. The purpose is to measure the effect of reduced tool reliability on queueing and cycle time.

Completed Lots 55	Average Cycle Time 16.847	Last Cycle Time 19.774	Average Queue Length 1.243
Most Utilized Machine Machine 1	Bottleneck Evidence One machine operated near full utilization, indicating potential bottleneck behavior.		

Figure 9: Increased downtime results

4.5 Summary of Experimental Findings

Across all scenarios, the digital twin showed clear sensitivity to changes in arrival pressure, recipe mix, and machine availability. The high-lot-load case produced the strongest overall degradation, with the largest average queue length and average cycle time, indicating that the photolithography step becomes severely congested when release volume greatly exceeds effective capacity. The recipe-focus case created a more moderate but still meaningful performance decline, showing that tool qualification can create imbalance even without extreme

demand. The increased-downtime case raised utilization across the tool set relative to baseline, but its effect on queueing and cycle time was smaller than the 10x load increase. As shown by the utilization table on page 11, Machine 1 was the most heavily used machine in every scenario, while Machines 4 and 5 remained lightly used except under heavier stress. Overall, the results show that the step is most sensitive to large demand surges, while recipe concentration and downtime create more targeted constraints that still reduce system flexibility.

Machine	Utilization (%)			
	Baseline	High Lot Load	Recipe Focus	Increased Downtime
1	50.8	82.3	82.8	68.4
2	34.9	74.6	69.3	53.5
3	42.2	68.1	31.2	55.4
4	5.8	32.6	8.6	34.3
5	5.8	27.3	0	24.7

Table 1: Machine utilization across scenarios

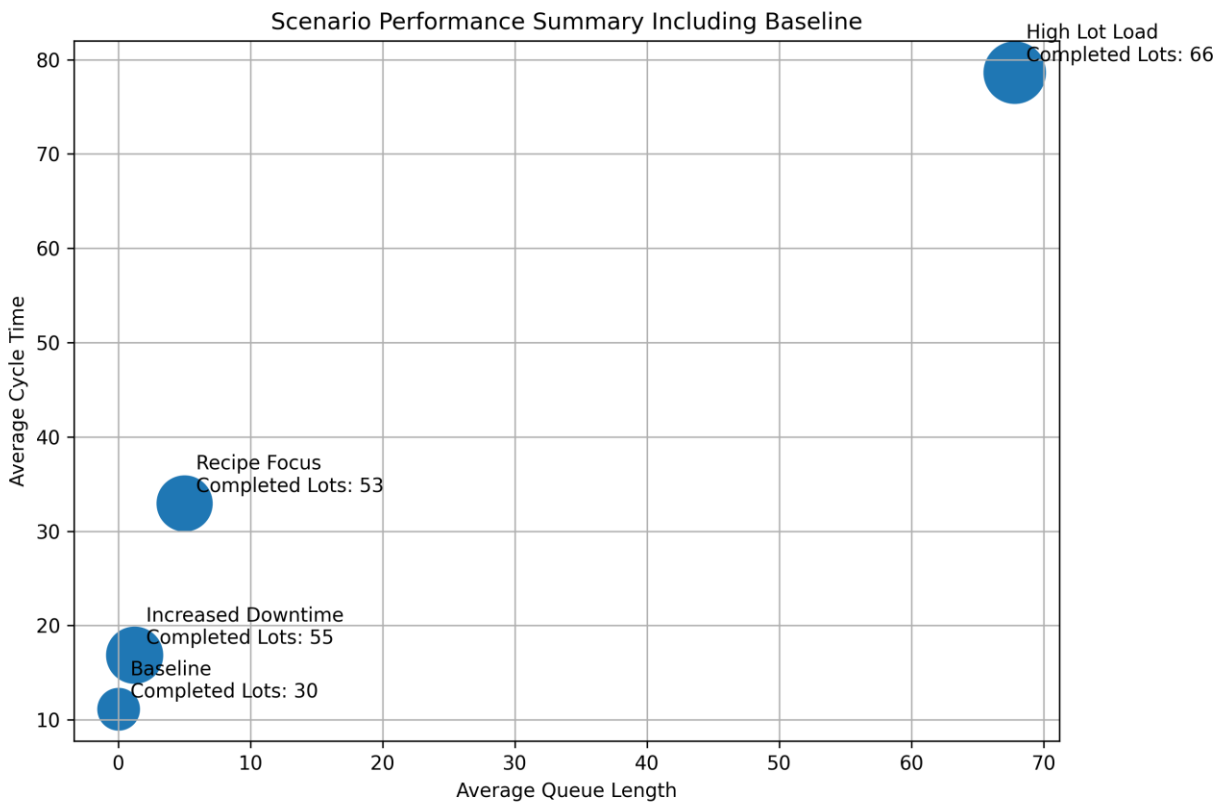


Figure 10: Performance comparison between the baseline run and each scenario run.

5. Discussion

The experimental results show that the digital twin responds differently to changes in demand, recipe mix, and downtime, which supports its usefulness as an analysis tool rather than only a visual simulation. The high-lot-load scenario had the largest impact on performance, producing the highest average queue length and average cycle time. This indicates that the photolithography step becomes heavily congested when lot arrivals greatly exceed effective capacity.

The recipe-focus scenario showed that recipe mix can also create meaningful stress even without extreme demand. Because not all machines are qualified for all recipes, concentrating arrivals on one recipe family shifts utilization toward a smaller subset of tools. The utilization table confirms this pattern, with Machines 1 and 2 carrying most of the load while other machines remain lightly used.

The increased-downtime scenario also reduced system flexibility, but its effect was smaller than the 10x lot-load increase under the tested settings. Overall, the results suggest that the model is most sensitive to large demand shocks, while recipe concentration and downtime create more targeted constraints. These outcomes are consistent with the intended logic of the twin and show that it can distinguish between different sources of operational stress.

6. Conclusion and Future Work

This project produced a working digital twin prototype of the photolithography step in semiconductor manufacturing. The model combines external lot generation in Python with an AnyLogic simulation engine, recipe-specific routing, machine-specific downtime, and runtime user controls. The resulting system can be used to test what-if scenarios involving demand, recipe mix, and equipment reliability while tracking performance measures such as queue length, cycle time, throughput, and utilization.

The main contribution of the project is the successful integration of data generation, data ingestion, simulation logic, and interactive scenario control into one framework. Although the model is simplified, the results show that it captures important operational behavior and can be used to compare different stress conditions in the photolithography step.

Future work should focus on replacing synthetic inputs with real fab data, expanding the process detail to include additional manufacturing constraints, and validating the twin against historical performance measures. With those additions, the model could support more realistic planning, analysis, and decision-making in an industrial setting.

In its current form, the project is best viewed as a strong prototype. It demonstrates the essential logic and infrastructure of a digital twin and provides a credible base for further development toward industrial relevance.